

Frontend Tech, Explained Like You're 10

Every library and file we used, why we picked it, and one line of the real code

This document is the technical companion to the Frontend Walkthrough. It goes tool by tool through the frontend of the Agentic Compliance Auditor. For every piece, it explains what the piece is, why we picked it over alternatives, a kid-simple analogy, and where it lives in our code.

The whole recipe (before we go tool by tool)

Building a website is like cooking a meal. The **ingredients** are libraries — bits of pre-made code we install. The **oven** is a build tool that bakes everything into something a browser can eat. The **plate** is the browser tab.

Here is the shopping list we chose:

Ingredient	Role in our meal
Vite	The oven — bakes the code into a website and reloads while you cook
React	LEGO blocks — every UI piece (button, panel) is a small block we snap together
TypeScript	Spellcheck — catches mistakes in the code before they hit the browser
Tailwind CSS	Sticker sheet — pre-made styles we paste on blocks to make them look nice
Zustand	Shared notebook — all blocks read and write to the same little memory pad
react-pdf	Picture frame for PDFs — shows the real document on the left half
pdf.js worker	Helper cook — chops the PDF in a separate kitchen so the main one stays fast
lucide-react	Icon stickers — the tiny pictures like arrows, checkmarks, paper airplane
clsx	Wardrobe manager — puts different clothes on a block based on its mood (state)

1. Vite

Purpose: The build tool that turns our code into a running website.

What it is

Vite is a program that watches the code files. When you save a file, Vite instantly rebuilds only that piece and pushes the change to the browser. When you're ready to publish, Vite bundles everything up into a small, fast package (in our project it shrinks a 550 KB app down to 167 KB after compression).

Why we picked it (and not something else)

The old-school alternative is Webpack, which is slow to start (30+ seconds) and complicated to configure. Vite starts in about 1 second and needs almost no config. Every modern React project starts with Vite unless there's a reason not to.

***Like a kid would say:** "Vite is the toy factory. You add a wheel to your LEGO car, and one second later you can already play with the new car on the screen."*

Where you see it in our code

- frontend/package.json — lists Vite as our tool.
- We start it with: `npm run dev`
- It's what serves the page at `http://localhost:5173` while you develop.

```
# start the dev server
npm run dev
```

```
# build the production version
npm run build
```

2. React

Purpose: The framework we build UI blocks with.

What it is

React lets you describe the screen as a set of small, reusable pieces called components. A button is a component. A whole panel is a component made of smaller components. React figures out what changed and only updates that part of the screen — so if only the answer text changes, only the answer area re-draws.

Why we picked it (and not something else)

React is the industry default. Every company hiring for AI apps expects it, so it doubles as a skill your GitHub proves. Alternatives (Vue, Svelte, Solid) are nice, but React has by far the biggest community and the best ecosystem for things like react-pdf that we needed.

Like a kid would say: "React is like LEGO. Every piece of the screen is a LEGO block. If the color of one block changes, you don't have to re-build the whole spaceship — just swap that one block."

Where you see it in our code

- Every file ending in .tsx in frontend/src/components/ is a React component.
- App.tsx snaps five components together to make the whole page.
- Each component is a function that returns what it should look like.

```
// frontend/src/App.tsx (simplified)
export default function App() {
  return (
    <div className="h-screen flex flex-col">
      <Header />
      <PdfViewer />    {/* left half */}
      <Sidebar />      {/* right half */}
    </div>
  )
}
```

3. TypeScript

Purpose: Spellcheck for the code.

What it is

TypeScript is JavaScript with labels. You say 'this variable holds a number', 'this function returns a list of chunks', and TypeScript checks — before the code runs — that everything matches up. It catches typos and shape mismatches at compile time instead of at midnight in production.

Why we picked it (and not something else)

For a portfolio project this matters twice over. First: less debugging, more shipping. Second: every serious company uses TypeScript. Showing that your code is typed and passes type-check is a signal to recruiters that you write real production code, not hobby scripts.

Like a kid would say: "TypeScript is your mom checking your homework before you turn it in. She catches the silly mistakes so the teacher doesn't have to."

Where you see it in our code

- types.ts — where we describe the shape of the events the backend will send us.
- Every component uses those types so we never accidentally read a field that doesn't exist.
- 'npm run build' fails if any type is wrong — meaning bad code never ships.

```
// frontend/src/types.ts
export interface RetrievedChunk {
  content: string
  page_number: number
  filename: string
  similarity_score: number
}
```

4. Tailwind CSS

Purpose: How we style everything without writing CSS files.

What it is

Instead of writing 'my-button { padding: 12px; background: blue; color: white }' in a separate file, Tailwind lets you write style directly on the block using short class names like px-3 (padding on the sides), bg-sky-500 (blue background), and text-white. It's the same styling — just written in a much faster, more consistent way.

Why we picked it (and not something else)

Traditional CSS files grow into a tangled mess as projects grow. Tailwind keeps every component self-contained: the block and its style live together. This is now standard at companies like Vercel, GitHub (parts of), OpenAI's site. It also pairs beautifully with React because you can compute the class names in code.

Like a kid would say: "Tailwind is a giant sticker sheet. Instead of painting your LEGO block from scratch, you pick a red-color sticker, a round-corners sticker, a big-font sticker, and slap them on."

Where you see it in our code

- tailwind.config.js — the settings file with our custom colors (sky, slate, panel).
- index.css — three lines that pull all of Tailwind's stickers into the app.
- Every className in every .tsx file — those short strings are Tailwind classes.

```
// From QuestionInput.tsx
<button className="px-3 rounded bg-sky-500 hover:bg-sky-400
  disabled:opacity-40 text-slate-900 font-medium">
```

Ask
</button>

5. Zustand (the shared memory)

Purpose: A tiny global notebook every component can read from and write to.

What it is

When the user types a question in the top-right, the answer needs to show up in a different component below. Instead of passing the question through every parent in between (painful), we put it in a shared store that any component can subscribe to. When the store changes, only components that use that piece re-render.

Why we picked it (and not something else)

The famous alternative is Redux — powerful but requires a lot of setup and ceremony (actions, reducers, middleware). Zustand does the same job in about 3 KB with a fraction of the code. React's own built-in Context API works too, but gets clunky when many components share many pieces of state. For an app our size, Zustand is the perfect middle ground.

Like a kid would say: "Zustand is the fridge magnet notepad. Anyone in the family can write on it. Anyone can read it. If mom changes the shopping list, everyone sees the new list the next time they look."

Where you see it in our code

- store.ts — defines the notebook: what fields it has, and functions to change them.
- Every component that needs data (PdfViewer, TracePanel, AnswerBox) calls useStore(...) to read from it.
- When you click Ask, runQuery() from the store starts the workflow and writes results into it as they arrive.

```
// frontend/src/store.ts (heart of it)
export const useStore = create<AppState>((set, get) => ({
  query: '',
  isStreaming: false,
  chunks: [],
  answer: '',
  activePage: null,
  jumpToPage: (page) => set({ activePage: page }),
  runQuery: async () => { /* streams events, updates store */ },
}))
```

6. react-pdf

Purpose: Displays a real PDF inside our page and lets us highlight text on it.

What it is

react-pdf wraps Mozilla's pdf.js library (the same PDF engine Firefox uses) in React components. You give it a PDF file path and a page number, it renders that page onto a canvas plus an invisible text layer on top. That text layer is what makes highlighting possible.

Why we picked it (and not something else)

The alternatives were: use an to embed the PDF (works but you can't highlight anything on it), or use a paid tool like Adobe's PDF Embed API (overkill and requires an account). react-pdf is free, works entirely in the browser, and lets us decorate the text — which is exactly what we need for the yellow citation highlights.

***Like a kid would say:** "react-pdf is a picture frame that also has an invisible layer of stickers above the picture. You can slap a yellow highlighter sticker over any word in the picture, and it looks like the word is highlighted."*

Where you see it in our code

- PdfViewer.tsx — imports Document and Page from react-pdf.
- The customTextRenderer prop is where we tell react-pdf 'wrap this text in when it matches a chunk'.
- The file prop is set to /GDPR.pdf, /HIPAA.pdf, or /SOC2.pdf depending on the active tab.

```
// PdfViewer.tsx — simplified
<Document file={`/${regulation}.pdf`}
  onLoadSuccess={({ numPages }) => setNumPages(numPages)}>
  <Page pageNumber={currentPage}
    width={700}
    customTextRenderer={({ str }) => highlight(str)} />
</Document>
```

7. The pdf.js worker

Purpose: The background helper that does the heavy PDF parsing.

What it is

Reading a PDF is heavy work — decoding fonts, laying out text, drawing shapes. If we did this on the browser's main thread, the whole page would freeze while a page loaded. The worker is a separate

JavaScript thread that does this heavy work in the background so the UI stays responsive.

Why we picked it (and not something else)

react-pdf requires a worker to function — this isn't really a choice, it's a configuration step. What matters is HOW we set it up. Many tutorials use a CDN (loading the worker file from unpkg.com), but that adds a network dependency and can break offline. We use Vite's ability to bundle the worker file locally, so everything comes from our own server.

***Like a kid would say:** "The worker is a helper cook in a second kitchen. The main cook (the UI) can keep smiling at guests while the helper chops the vegetables (pares the PDF) in the back."*

Where you see it in our code

- Top of PdfViewer.tsx — one line that tells pdf.js where to find the worker file.
- The worker itself lives in node_modules/pdfjs-dist/build/pdf.worker.min.mjs.

```
// PdfViewer.tsx (top of file)
import { pdfjs } from 'react-pdf'
pdfjs.GlobalWorkerOptions.workerSrc = new URL(
  'pdfjs-dist/build/pdf.worker.min.mjs',
  import.meta.url,
).toString()
```

8. lucide-react (icons)

Purpose: The little pictures — arrows, checkmarks, paper airplane, document icon.

What it is

lucide-react is a collection of hand-drawn icons packaged as React components. Each icon is a tiny SVG shape you use like a component: `<Icon />`. You can change the size and color with props.

Why we picked it (and not something else)

The alternative — importing icon fonts like Font Awesome — is slower to load (pulls in hundreds of icons you don't use) and harder to style. lucide-react only bundles the icons you actually use, and each icon is a small, sharp vector that scales cleanly at any size.

***Like a kid would say:** "lucide is your sticker book of tiny drawings. When you want a checkmark, you peel off a checkmark sticker. When you want an arrow, you peel off an arrow."*

Where you see it in our code

- TracePanel.tsx — Check (green tick), Loader2 (spinning spinner), Circle (empty).
- QuestionInput.tsx — Send (paper airplane).
- PdfViewer.tsx — FileText, ChevronLeft, ChevronRight.

```
import { Check, Loader2, Circle } from 'lucide-react'
```

```
// Then use them like components:  
<Check size={16} className="text-green-400" />  
<Loader2 size={16} className="animate-spin" />
```

9. clsx (conditional classes)

Purpose: Combines class names based on the block's current mood (state).

What it is

Often a component looks different in different states — the active tab is blue, the inactive tabs are grey. Instead of writing messy if/else statements to build the className string, clsx lets you list conditions and it stitches them together.

Why we picked it (and not something else)

It's a 200-byte utility that makes styling code readable. The plain-JavaScript alternative is string concatenation with ternary operators, which quickly becomes unreadable. Every mature React project uses clsx or its cousin classnames.

***Like a kid would say:** "clsx is a wardrobe manager. You tell it 'if it's sunny, wear the shorts; if it's raining, add the raincoat', and it hands you the right outfit."*

Where you see it in our code

- RegulationTabs.tsx — highlights the active tab differently from the inactive ones.
- TracePanel.tsx — colors each trace step differently based on pending/active/done.

```
// RegulationTabs.tsx  
<button className={clsx(  
  'px-4 py-1.5 text-sm rounded',  
  regulation === opt  
    ? 'bg-sky-500/20 text-sky-300 font-medium'  
    : 'text-slate-400 hover:text-slate-200'  
)}>{opt}</button>
```

10. The mock event stream (mockStream.ts)

Purpose: A pretend AI that lets us build the UI before the real AI is ready.

What it is

The real backend eventually streams events like 'retrieved 3 chunks', 'generating', 'answer ready'. Because your machine can't run the real backend yet, we wrote a fake generator that yields those same events on a timer — so the UI thinks it's talking to a real AI.

Why we picked it (and not something else)

Two reasons. (1) You can build and demo the frontend today, not next week. (2) When the real backend comes online, swapping mock for real is a one-line change — because both produce the same shape of events. This is the same 'mock first, wire later' technique used in professional software teams.

***Like a kid would say:** "mockStream is a puppet that pretends to be the AI. It says the same lines the real AI would say, just so we can practice the play before the real actor arrives."*

Where you see it in our code

- mockStream.ts — an async generator that yields fake events with delay() between them.
- store.ts imports it and consumes it exactly like it will consume the real stream.
- Later we'll add realStream.ts that hits POST /query/stream — swap in one place.

```
// mockStream.ts (heart)
export async function* mockEventStream(query: string) {
  yield { event: 'start', data: { session_id: '...' } }
  await delay(400)
  yield { event: 'retrieved', data: { chunks: [...] } }
  await delay(900)
  yield { event: 'generating', data: {} }
  /* ... */
}
```

Where does everything live? (folder tour)

Here is what's inside frontend/ and what each file does. If you ever want to change one specific thing on the screen, this table tells you which file to open.

File	What it does
frontend/index.html	The empty HTML page the app is injected into.
frontend/vite.config.ts	Vite settings (mostly defaults).
frontend/tailwind.config.js	Our custom colors + which files to scan for Tailwind classes.
frontend/postcss.config.js	Boilerplate so Tailwind runs when Vite builds.
frontend/public/GDPR.pdf	The GDPR document served straight to the PDF viewer.
frontend/public/HIPAA.pdf	Same, for HIPAA.
frontend/public/SOC2.pdf	Same, for SOC2.
src/main.tsx	The entry point — hands the <App /> block to the browser.
src/App.tsx	The top-level layout: header + two-panel body.
src/index.css	Global styles — background color, PDF highlight color.
src/types.ts	Shape descriptions for events, chunks, trace steps.
src/store.ts	The shared notebook (Zustand). Question, answer, chunks, active page all live here.
src/mockStream.ts	Fake AI that yields the event sequence for local dev.
src/components/RegulationTabs.tsx	The GDPR / HIPAA / SOC2 pill buttons in the header.
src/components/QuestionInput.tsx	The text box + Ask button at the top of the sidebar.
src/components/PdfViewer.tsx	The PDF display + page navigation + citation highlighting.
src/components/TracePanel.tsx	The three-step live checklist (Retrieve → Generate → Reflect).
src/components/AnswerBox.tsx	The answer text with clickable citation chips + source list.

What happens when you click 'Ask'?

Follow the arrow. Each step names the file and the piece of code doing the work.

- 1. You type in the input box — `QuestionInput.tsx` calls `setQuery(text)` on the store.
- 2. You press Enter — `QuestionInput.tsx` calls `runQuery()` on the store.
- 3. `store.ts` runs `runQuery()`: it flips `isStreaming = true` and starts iterating events.
- 4. Each event is passed to `applyEvent()` which updates the right piece of the store.
- 5. `TracePanel.tsx` notices the trace array changed → re-renders the checklist.
- 6. `AnswerBox.tsx` notices `answer + chunks` changed → renders the answer with citation chips.
- 7. You click a chip — `AnswerBox.tsx` calls `jumpToPage(chunk.page_number)`.
- 8. `PdfViewer.tsx` notices `activePage` changed → calls `setCurrentPage` → `react-pdf` renders that page.
- 9. `customTextRenderer` runs on every text bit → wraps matching substrings in `.`
- 10. Your CSS in `index.css` paints those marks yellow. The passage lights up.

The takeaway

None of these tools is exotic. Every one of them is what a working React developer at a real company would reach for. What you can genuinely say on your CV: **"Built a React + TypeScript + Vite frontend with a Zustand store, streaming event handler, and react-pdf-based document annotator."** Every word of that is true, and every choice on this document backs it up.